

ESC112 Tutorial 4

1 Introduction

Topics to be covered:

- dynamic memory allocation
- multi dimensional arrays
- pointer to pointers
- application to matrices and images

Codes etc files can be found in the hello IITK folder for tutorial 4 under resources.

2 Dynamic memory allocation

there are 4 functions in `stdlib.h` to help with memory allocation.

```
1 void *calloc(size_t nobj, size_t size)
```

- `calloc` returns a pointer to space for an array of `nobj` objects, each of size `size`, or `NULL` if the request cannot be satisfied. The space is initialized to zero bytes.

```
1 void *malloc(size_t size)
```

- `malloc` returns a pointer to space for an object of size `size`, or `NULL` if the request cannot be satisfied. The space is uninitialized.

```
1 void *realloc(void *p, size_t size)
```

- `realloc` changes the size of the object pointed to by `p` to `size`. The contents will be unchanged up to the minimum of the old and new sizes. If the new size is larger, the new space is uninitialized. `realloc` returns a pointer to the new space, or `NULL` if the request cannot be satisfied, in which case `*p` is unchanged.

```
1 void free(void *p)
```

- `free` deallocates the space pointed to by `p`; it does nothing if `p` is `NULL`. `p` must be a pointer to space previously allocated by `calloc`, `malloc`, or `realloc`.

safe practices:

- always allocate a pointer before using. use sizeof and not fixed bytes. we never know if int is really 4 bytes in a particular machine.

```
1 int *a = malloc(sizeof(int));
2 //auto type casted from void * to variable's type
```

- always check for NULL after

```
1 int *a = calloc(1, sizeof(int));
2 if(a == NULL)
3     exit(1);
4 //calloc makes *a = 0
5 *a = 42;
```

- always assign realloc return value to a temp pointer and check temp pointer for NULL. if not, only then assign to desired pointer. directly assigning to desired pointer may lose old pointer, if NULL is returned.

```
1 int *a = malloc(sizeof(int));
2 if(a == NULL)
3     exit(1);
4 ...
5 int *temp = realloc(10, sizeof(int));
6 //need not extend a, previous data copied though
7 if(temp == NULL){
8     //can't reallocate
9 }
10 else{
11     a = temp;
12 }
```

- free a pointer as soon as it is needed no more. after freeing set it to NULL.

```
1 free(a);
2 a = NULL;
```

- always free before doing repeated malloc on same pointer

```
1 int *a = calloc(1, sizeof(int));
2 if(a == NULL)
3     exit(1);
4 free(a);
5 a = malloc(10 * sizeof(int));
```

3 static multi-dimensional arrays

- C allows us to allocate arrays of two or more dimensions.

```
1 int chessboard[8][8];
```

- individual elements are accessed as `chessboard[i][j]`
- one can initialize or set a 2-D array as follows.

```
1 int v_sobel[3][3] = {
2     {-1, 0, 1},
3     {-2, 0, 2},
4     {-1, 0, 1}
5 };
```

basically, braces inside braces hierarchy until no more dimensions are left.

- internally an array such as

```
1 int chessboard[8][8];
```

is flattened and stored as an 1D array of size 64. `chessboard[2][3]` is mapped to index $2 * 8 + 3 = 19$ in the 1D array. That is why both of the following will print 2.

```
1 #include <stdio.h>
2
3 int main() {
4     int v_sobel[3][3] = {
5         {-1, 0, 1},
6         {-2, 0, 2},
7         {-1, 0, 1}
8     };
9     printf("%d_%d", v_sobel[1][2], *(v_sobel + 5));
10 }
```

`v_sobel` decays to pointer to first row `int (*)[3]`. `*v_sobel` is the first row array, i.e., `int[3]` – decays to pointer to first element `&v_sobel[0][0]`.

So `*v_sobel + 5` is pointer arithmetic – moves 5 ints forward from `v_sobel[0][0]`. `*(v_sobel + 5)` dereferences to 6th element in linear memory.

- we need to be careful about passing multidimensional matrices to arrays. the function must know how to calculate this 2D to 1D mapping or in general nD to 2D mapping. it basically needs all but the first dimension's size. The following are all valid function definitions involving 2D arrays.

```
1 void f(int temp[][3]) {}
2 int f(int temp[3][3]) {}
3 int f(int (*temp)[3]) {}
```

recall for 1D arrays, we could simply do `f(int temp[])`. An alternative is to pass the size as well: `f(int temp[10])`

the `()` in the last line is needed. `int (*temp)[3]` and `int *temp[3]` are completely different things.

- the former is a pointer to a an array of 3 integers. the second is an array of int pointers of size 3.

- in the former, `temp + 1` will skip 3 integers (i.e 12 bytes if `int` is 4 bytes). in the later, `temp + 1` will skip 1 integer pointer (i.e. 4 or 8 bytes etc depending on how large addresses are).
- similar concepts of flattening and argument-passing extend to 3 or more dimensional arrays. only first dimension could be `[]`, others must be specified.

4 dynamic multi dimensional arrays

Pointers are just another variables (that can holds addresses of proper data types). So, we can take addresses of pointers themselves. what we get as a result is a pointer to pointer. and we can take addresses of them as well, and so on to get `(int *) *` or `int **`, `int ***` etc.

```
1 int x = 2, *p = &x, **q = &p;
2 printf("%d_%d_%d_p_p_p", x, *p, **q, *q, p, q);
3 //2 2 2 0x7ffc1759bbe4 0x7ffc1759bbe4 0x7ffc1759bbe8
```

Here, `q` has type `int **` (address). `*q` and `p` have types `int *` (address). `x`, `*p`, and `**q` have types `int` (number).

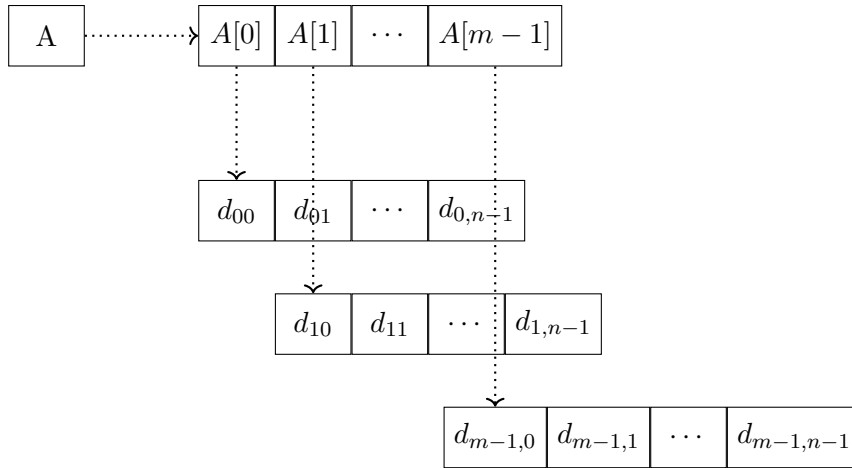
instead of static allocation such as `int chessboard[8][8]`; we can allocate it dynamically one dimensional after another.

```
1 int ** chessboard = malloc(8 * sizeof(int *));
2 if(chessboard == NULL)
3     exit(1);
4 for(int i = 0; i < 8; i++){
5     chessboard[i] = malloc(8 * sizeof(int));
6     if(chessboard[i] == NULL){
7         for(int j = 0; j < i; j++)
8             free(chessboard[j]);
9     }
10    free(chessboard);
11    exit(1);
12 }
```

First one will be typecasted to `int **` and the later ones to `int *`. after using this matrix, we can free the rows first and then the matrix.

```
1 for(int i = 0; i < 8; i++)
2     free(chessboard[i]);
3 free(chessboard)
```

Such a dynamic allocation will not be flattened, unlike previous section. Basically, `chessboard` is the starting address of an array of 8 `int *` variables. each the later variables in turn holds the starting addresses of a row of 8 integers. therefore, it can be simply passes to a function as `int **`. function don't need the number of columns unlike the static case, because no remapping to 1D is done and everything is happening via addresses after proper allocation. See the illustration for an `m` by `n` dynamically allocated matrix.



```
1 f(int **A) { }
```

5 Applications: matrix algebra

2D arrays are basically matrices, we can do familiar computations such as multiplications determinants, transpose. Here is a complete code for some operations.

Recall matrix multiplications.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} (1 \cdot 5 + 2 \cdot 7) & (1 \cdot 6 + 2 \cdot 8) \\ (3 \cdot 5 + 4 \cdot 7) & (3 \cdot 6 + 4 \cdot 8) \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

- $C = AB$. A's number of columns = B's number of rows
- A is $m \times n$. B is $n \times p$. Then, C is $m \times p$.
- $C[i][j]$ can be obtained by multiplying i-th row of A with j-th column of B element wise and taking their sum.

multiplications determinants, transpose extends naturally for m by n matrices.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 double getdouble(){
5     double d;
6     scanf("%lf", &d);
7     return d;
8 }
9
10 double **alloc_matrix(int m, int n){
11     double **A = malloc(m * sizeof(double *));
12     if(A == NULL)
13         exit(1);
14     for(int i = 0; i < m; i++){
15         A[i] = malloc(n * sizeof(double));
16         if(A[i] == NULL){
17             for(int j = 0; j < i; j++)
18                 free(A[j]);
19             free(A);
20             exit(1);
21         }
22     }
23     return A;
24 }
25
26 void read_matrix(double **A, int m, int n){
27     for(int i = 0; i < m; i++)
28         for(int j = 0; j < n; j++)
29             A[i][j] = getdouble();
30 }
31
32 void mat_mul(double **A, double **B, double **C, int m, int n, int p){
33     //A, B are m X n and n X p
34     //C = A X B is m X p
35     //pick i-th row of A and j th column of B
36     //multiply element wise and add
37     //set it to C[i][j]
38     double sum;
39     for(int i = 0; i < m; i++){
40         for(int j = 0; j < p; j++){
41             sum = 0;
42             for(int k = 0; k < n; k++)
43                 sum += A[i][k] * B[k][j];
44             C[i][j] = sum;
45         }
46     }
47 }

```

```

1 void print_matrix(double **M, int m, int n){
2     for(int i = 0; i < m; i++){
3         for(int j = 0; j < n; j++){
4             printf("%lf_", M[i][j]);
5             printf("\n");
6         }
7     }
8
9 void free_matrix(double **M, int m){
10     for(int i = 0; i < m; i++){
11         free(M[i]);
12     }
13     free(M);
14 }
15
16 int main(){
17     int m, n, p;
18     //m X n, n X p
19     scanf("%d_%d_%d", &m, &n, &p);
20
21     //allocate and read A
22     double **A = alloc_matrix(m, n);
23     read_matrix(A, m, n);
24
25     //allocate and read B
26     double **B = alloc_matrix(n, p);
27     read_matrix(B, n, p);
28
29     //allocate and compute C = A X B
30     double **C = alloc_matrix(m, p);
31     mat_mul(A, B, C, m, n, p);
32
33     //print the product matrix
34     print_matrix(C, m, p);
35
36     //free the matrices
37     free_matrix(A, m);
38     free_matrix(B, n);
39     free_matrix(C, m);
40 }

```

Remarks:

- notice alloc matrix functions takes m and n as input allocates the m by n matrix and returns it as double **. instead we can't declare a double **M; and pass in into alloc matrix. it will only receive a copy of M as usual and allocate it instead of M. exactly what happens in swap using variables instead of address. if we really want that way, we need to pass &M and then the first argument of alloc matrix can be double ***A. in that case, inside alloc matrix function, we need to use *A in place of A everywhere.

```

1 double **M;
2 alloc_matrix(&M, m, n);

```

- notice read matrix takes a double ** as argument to write into the matrix. these needs to be properly allocated as discussed before. otherwise, runtime error.

The above has also been covered in the class and can be discussed faster.

5.1 Determinant using Recursive Expansion (Laplace Expansion)

We can compute the determinant of an $n \times n$ matrix using a simple recursive idea.

The idea is to expand along the first row.

Example: 3×3 Matrix

Consider the matrix:

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

We expand along the first row:

$$\det(A) = a \cdot \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \cdot \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \cdot \begin{vmatrix} d & e \\ g & h \end{vmatrix}$$

Step-by-step idea

For each element in the first row:

- Remove its row and column
- Compute the determinant of the smaller matrix
- Multiply with the element
- Add or subtract based on position

The signs follow this pattern:

+ - +

Recursive Idea

The same idea extends for $n > 3$ as well. To compute the determinant of an $n \times n$ matrix:

- Expand along the first row
- For each element:
 - Create an $(n - 1) \times (n - 1)$ matrix
 - Recursively compute its determinant

Base Case

If the matrix is 1×1 , then:

$$\det([x]) = x$$

Here is the C code to compute determinants based on Laplace expansion – an extension of the previous code. we allocate the submatrices after removing the first row and each of columns, compute its determinant using a recursive call, combine the answers with proper signs and coefficients and then free the submatrices.

```
1 double determinant(double **M, int n){
2     if(n == 1)
3         return M[0][0];
4
5     double sum = 0, sign = 1.0;
6
7     //expand along the first row
8     for(int i = 0; i < n; i++){
9         double **N = alloc_matrix(n - 1, n - 1);
10        //copy all but index 0 row and index i column
11        for(int j = 1; j < n; j++){
12            //copy all columns up to column index i - 1
13            for(int k = 0; k <= i - 1; k++){
14                N[j - 1][k] = M[j][k];
15            }
16            //copy all columns index i + 1 or more
17            for(int k = i + 1; k < n; k++){
18                N[j - 1][k - 1] = M[j][k];
19            }
20        }
21        sum += sign * M[0][i] * determinant(N, n - 1);
22        free_matrix(N, n - 1);
23        sign *= -1;
24    }
25    return sum;
26 }
```

Determinants are only defined for square matrices. To test this code, just add a line in the main function of the previous code. This also verifies the identity, $\det(AB) = \det(A) \cdot \det(B)$.

```
1 if(m == n && n == p){
2     printf("detA:_%lf,_detB:_%lf,_detC:_%lf\n", determinant(A, m),
3         determinant(B, m), determinant(C, m));
4 }
```

Time Complexity

This method is simple but slow.

$$T(n) \approx n \cdot T(n - 1) \Rightarrow O(n!)$$

So it works well only for small matrices.

Remark

In practice, faster methods based on row operations can compute determinants in $O(n^3)$ time, but they are more advanced.

Exercise

Rewrite the code to expand along any other row or column and verify that determinant stays the same.

6 Application: simple image processing

in the pre deep learning times, filtering techniques were quite successful and popular way to process an image. This exercise illustrates an appropriate use of 2D arrays for implementing such a filtering technique.

Images are basically matrices. Each pixel is an entry of this matrix. A color image has 3 matrices or 3 channels for R, G, B. Instead, we will work with grayscale images for simplicity, where each pixel has a value between 0 to 255 say. An ascii based format for storing images is the .pgm format. A pgm image such as shown in in figure 1 can be read in a text editor and it looks as follows.



Figure 1: Input cameraman image for edge detection

```

1 P2
2 256 256
3 255
4 156 159 158 155 158 156 159 158 157 158 158 159 160 160 160 158 163 161
   162 160 164 160 165 163 161 163 161 164 165 162 161 165 165 164 166
   165 164 166 167 165 165 164 170 166 167 167 170 168 169 166 169 167
   170 169 169 171 169 171 169 171 170 169 173 173 172 171 173 176 174
   176 180 179 178 178 181 181 181 177 180 178 179 180 180 180 178 184
   181 182 183 182 183 181 182 183 182 185 184 181 181 183 184 183 184
   183 186 186 187 184 183 185 185 185 182 183 186 182 184 187 185 188
   185 185 186 184 184 182 186 186 185 181 186 182 182 185 183 180 181
   182 181 185 182 186 182 181 183 182 181 181 181 179 181 177 180 181
   178 178 180 179 180 177 179 180 178 176 178 174 177 178 173 177 177
   176 172 172 172 176 174 177 173 173 169 172 169 171 167 167 166 167
   168 168 168 168 169 166 169 165 166 166 166 164 166 169 166 168 166
   165 168 167 168 165 166 166 164 163 164 159 164 164 158 162 162 162
   161 159 161 161 159 159 160 155 155 154 155 153 156 155 158 156 155
   155 156 154 151 156 155 152 155 155 155 153 156 154 153 151 152 152
5 ...
6 255 more rows

```

the first line say P2 to denote the PGM ascii format. the next line has two numbers for height and width. next line has the max value of any pixel 255. then the 256 rows start, each having 256 columns of integets between 0 to 255 denoting the grayscale value of the pixel. 0 means black, 255 means white, larger means lighter color.

Now, we wil use the following two filters defined as follows.

```

1          //define filters
2 int v_sobel[3][3] = {
3     {-1, 0, 1},
4     {-2, 0, 2},
5     {-1, 0, 1}
6 };
7 int h_sobel[3][3] = {
8     {-1, -2, -1},
9     {0, 0, 0},
10    {1, 2, 1}
11 };

```

both are 3 X 3 matrices. v_sobel helps us to detect vertical edges and h_sobel helps us to detect horizontal edges.

at (i,j)-th pixel of the original image we will multiply the 3 X 3 sub-matrix defined from the (i-1,j-1) pixel till the (i+1, j+1) pixel term by term with the above two filters. see Figure 2 for an illustration.

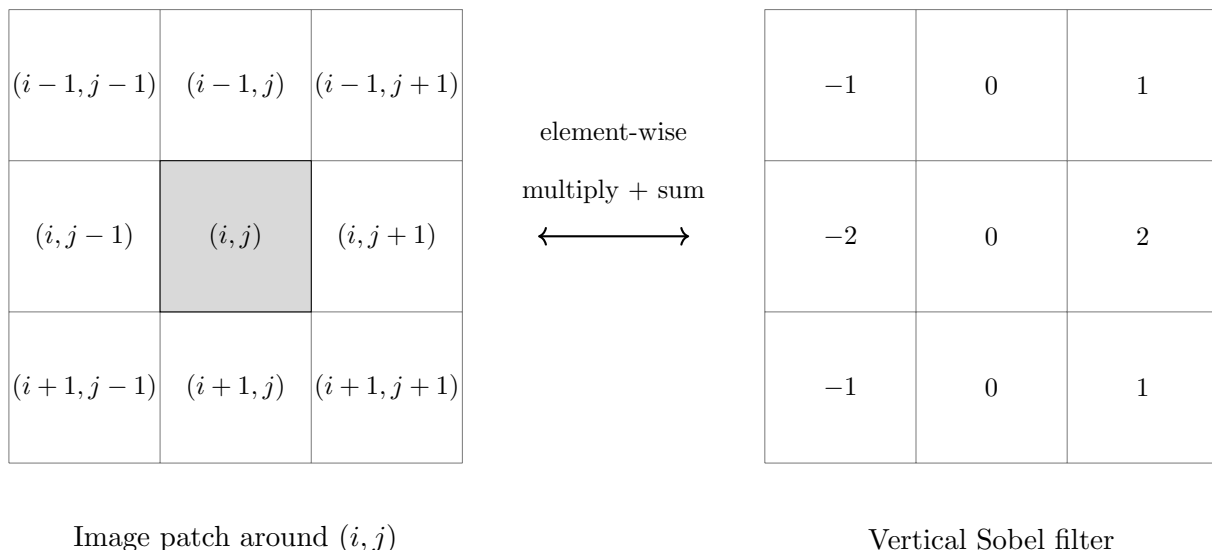


Figure 2: vertical edge detection using sobel filter. 9 pixel matrix centered around pixel (i, j) is multiplied element wise with the filter and then summed up. if there is a vertical edge, this will have a high value. horizontal edge detection is similar.

the numbers are chosen in a manner that whenever there is a jump in brightness values left to right, `v_sobel` will have large magnitude – either positive or negative and whenever there is a jump in brightness values top to bottom, `h_sobel` will have large magnitude. if v_{ij} and h_{ij} are these two magnitudes, the final output value at (i, j) is defined as $\sqrt{v_{ij}^2 + h_{ij}^2}$. This value will be high for edges and low for non-edges.

note that if our image is $m \times n$ pixels, and we would like to get the output also $m \times n$, we need to take care of the boundary and corner cases where there could be 1 or 2 missing rows. to take care of this, we just pad the pictures with 0s 1 pixel wide in all directions to get an $(m + 2) \times (n + 2)$ image.

Assume the image is given in pgm format and given in the standard input. The following C code generates the output after applying filters as discussed above. In the output, any number larger than 255 is changed to the maximum value 255. Larger values indicate presence of an edge and will be shown in light colors closer to white.

```

1 #include <stdio.h>
2 #include <math.h>
3 #include <stdlib.h>
4
5 void apply_sobel(int **image, int height, int width, int v_sobel[][3], int
    h_sobel[][3], int i, int j, int *v_out, int *h_out){
6     if(i > height || i < 1 || j > width || j < 1)
7         return; //illegal
8
9     //apply filters: align filter 1, 1 with i, j
10    *v_out = *h_out = 0;
11    for(int k = 0; k < 3; k++){
12        for(int l = 0; l < 3; l++){
13            *v_out += v_sobel[k][l] * image[i + k - 1][j + l - 1];
14            *h_out += h_sobel[k][l] * image[i + k - 1][j + l - 1];
15        }
16    }
17 }
18
19 int **allocate_matrix(int height, int width){
20     int **M = calloc(height, sizeof(int *));
21     for(int i = 0; i < height; i++)
22         M[i] = calloc(width, sizeof(int));
23     return M;
24 }
25
26 void read_image(int **M, int height, int width){
27     //fills 1 to height and 1 to width.
28     //0 height + 1, width + 1 will remain 0 padded
29     for(int i = 1; i <= height; i++){
30         for(int j = 1; j <= width; j++){
31             scanf("%d", &M[i][j]);
32         }
33     }
34 }
35
36 int main(){
37     char type[3]; //P2 or P3
38     scanf("%s", type); ///s stops at whitespace
39     int height, width;
40     scanf("%d_%d", &height, &width); //pixels
41     int maxval;
42     scanf("%d", &maxval);
43
44     //print the headers
45     printf("%s\n%d_%d\n", type, height, width, maxval);
46
47     //allocate image and filter output
48     //image is 1 pixel more in all direction for 0 padding
49     //calloc does 0 padding by default
50
51     int **image = allocate_matrix(height + 2, width + 2);
52     int **output = allocate_matrix(height, width);

```

```

1  //read image
2  read_image(image, height, width);
3
4  //define filters
5  int v_sobel[3][3] = {
6      {-1, 0, 1},
7      {-2, 0, 2},
8      {-1, 0, 1}
9  };
10 int h_sobel[3][3] = {
11     {-1, -2, -1},
12     {0, 0, 0},
13     {1, 2, 1}
14 };
15
16 //apply filters
17 int v_out, h_out;
18 for(int i = 1; i <= height; i++){
19     for(int j = 1; j <= width; j++){
20         apply_sobel(image, height, width, v_sobel, h_sobel, i, j, &
v_out, &h_out);
21         output[i-1][j-1] = (int) sqrt(h_out * h_out + v_out * v_out);
22         printf("%d_", output[i-1][j-1] > maxval? maxval : output[i-1][
j-1]);
23     }
24     printf("\n");
25 }
26
27 //free matrices
28 for(int i = 0; i < height + 2; i++)
29     free(image[i]);
30 free(image);
31
32 for(int i = 0; i < height; i++)
33     free(output[i]);
34 free(output);
35 }

```

Remarks:

- we are using one extra rows each in the top and bottom and one extra column each in left and right. zero padding happens by default using calloc.
- sobel filter are passed as static array and therefore, row size is needed in apply_sobel function. input and output images are dynamically allocated and hence can be simply passed as int **. also note the implicit return of the filter outputs for each pixel.
- math.h has all sorts of mathematical functions, usually returns double. sqrt is for square root. whenever math.h is used, you need to use the gcc -lm flag to compile in linux.
- while experimenting you may use ./a.out < input.pgm > output.pgm to redirect stdin to input.pgm and stdout to output.pgm. or, in prutor you may simply copy-paste the contents of input.pgm into the input box and run the program, and then copy the output

into output.pgm. note that prutor can only handle small size image outputs say less than 100 by 100.

- scanf with %s reads a sequence of non-whitespace characters – no & needed before char array variable name. It stops reading at the first whitespace (space, tab, newline). It automatically appends a null character (\0) at the end.

The output can be saved in .pgm format as well. pgm files can be opened as a picture or can be converted to/from other formats such as png by online and offline image processing softwares¹. The output for the above images is shown in Figure 3 and the edges have been detected quite successfully.



Figure 3: Cameraman output image from edge detection

¹for example the default image viewer in ubuntu.